TP 1 : Test de rentrée.

L'intégralité du TP est à réaliser en Ocaml. Ne cherchez pas de solutions algorithmiques trop compliquées, ce TP a pour but de vous remettre dans le bain du Ocaml et de l'algorithmie en général.

Préliminaires

Si $a > 0$ et $b > 0$ sont deux entiers, on note $a \mod b$ le reste de la division euclidienne de a par b, autrement dit l'unique entier r avec $0 \leq r < b$ tel qu'il existe un entier q satisfaisant $a = bq + r$.

Ce sujet ne porte que sur des graphes non orientés, chaque utilisation du mot graphe doit être entendue comme graphe non orienté. Un graphe G est un couple (S, A) où S est l'ensemble fini des sommets et A est l'ensemble des arêtes. Chaque arête est un ensemble de cardinal 2. Cette définition interdit donc les boucles.

Générateur de nombres pseudo-aléatoires

Étant donné u_0, on définit la suite des (u_n) par récurrence :

$$\forall n, u_{n+1} = (900\,007 \times u_n) \mod 1\,000\,000\,007$$

Vous avez à votre disposition les réponses à toutes les questions pour $u_0 = 56$. À vous de déterminer les valeurs pour $u_0 = 17$.

Question 1 : Calculer les valeurs suivantes :

- $u_1 \mod 1000$
- $u_{42} \mod 1000$
- $u_{1000} \mod 1000$
- $u_{10^6} \mod 1000$

Correction :
J'ai choisi de remplir un tableau afin de ne calculer une seule fois chaque valeur de u. Ainsi, si j'ai besoin de celles-ci plus tard, il sera possible de simplement lire la case correspondante du tableau.

```
(* Fonction pour créer un tableau contenant les termes de la suite u
   jusqu'à n termes *)
let tableau_u u0 n =
  (* Initialisation d'un tableau de taille n avec tous les éléments é
     gaux à u0 *)
  let t = Array.make n u0 in
  (* Fonction récursive pour calculer chaque terme de la suite et le
     stocker dans le tableau *)
  let rec aux i =
    if i = n-1 then ()  (* Arrêt si on atteint le dernier élément *)
    else begin
      (* Calcul du terme suivant dans la suite avec la récurrence donné
         e *)
      t.(i+1) <- (900007 * t.(i)) mod 1000000007;
      aux (i+1)  (* Appel récursif pour remplir le tableau *)
    end
  in aux 0; t  (* Démarre la récursion avec i=0 et retourne le tableau
     t *)

(* Stockage de la suite u jusqu'à u_1000000 *)
let t = tableau_u u0 1000001

(* Fonction pour accéder au n-ième terme de la suite u *)
let terme_u n = t.(n)
```

Élimination de doublons dans une liste d'entiers

Il existe différentes méthodes pour éliminer les doublons d'une liste. Dans ce sujet nous allons présenter une méthode qui peut être efficace sur les listes composées d'entiers naturels où l'on connaît une borne sur l'entier le plus grand présent dans la liste.

Algorithme 1 ÉliminеDoublon(l)

1: **Entrée :** Une liste l d'entiers dans l'intervalle $[0, N-1]$, avec $N \leq N_{\max}$.
2: **Sortie :** Une liste sans doublon.
3: **Initialisation :** Le tableau `estDoublon` est supposé déjà alloué à la taille $N_{\max} = 100000$.
4: **pour** chaque $i \in l$ **faire**
5: `estDoublon`$[i] \leftarrow$ `Faux`
6: **fin pour**
7: `listeSansDoublon` $\leftarrow$ liste vide
8: **pour** chaque $i \in l$ **faire**
9: **si** non `estDoublon`$[i]$ **alors**
10: Ajouter i à `listeSansDoublon`
11: `estDoublon`$[i] \leftarrow$ `Vrai`
12: **fin si**
13: **fin pour**
 return `listeSansDoublon`

On note $L(M) = \{u_i \mod M | 0 \leq i < M\}$.

Question 2 : Calculer le cardinal de l'ensemble $L(M)$ pour les valeurs de M suivantes :

- $M = 10$
- $M = 5\,432$
- $M = 98\,765$

Correction :
On implémente l'algorithme proposé en définissant `estDoublon` à l'extérieur de la fonction. On fait notamment attention à la syntaxe `if test then c1; c2`, le parenthésage par défaut étant : `(if test then c1); c2`.

```
(* Création d'un tableau pour marquer les doublons *)
let estDoublon = Array.make 100000 false

(* Fonction pour éliminer les doublons dans une liste *)
let elimineDoublon l =
  (* Réinitialisation des marquages dans le tableau estDoublon *)
  let rec init l = match l with
    | [] -> ()
    | h::q -> estDoublon.(h) <- false; init q
  in init l;
  (* Ajoute chaque élément de la liste s'il n'est pas déjà marqué *)
  let rec ajouter acc l = match l with
    | [] -> acc
    | h::q when not estDoublon.(h) -> estDoublon.(h) <- true;
                                      ajouter (h::acc) q
    | h::q -> ajouter acc q
  in ajouter [] l
```

Pour obtenir les résultats recherchés, on calcule tous les termes $u_i \mod M$ nécessaires que l'on stocke dans une liste. On renvoie celle-ci après en avoir éliminé les doublons. Enfin, on en calcule la longueur à l'aide d'une fonction classique.

```
(* Fonction pour créer l'ensemble L(m) *)
let ensemble_L m =
  (* Remplir une liste avec les termes u_i mod m *)
  let rec remplir i = match i with
    | -1 -> []  (* Arrêt quand i atteint -1 *)
    | i -> (t.(i) mod m)::(remplir (i-1))  (* Ajoute u_i mod m à la
       liste *)
  in elimineDoublon (remplir (m-1))  (* Élimine les doublons de cette
     liste *)

(* Fonction pour calculer la longueur d'une liste *)
let rec longueur l = match l with
  | [] -> 0  (* Cas de base : la liste est vide *)
  | h::q -> 1 + (longueur q)  (* Ajoute 1 pour chaque élément *)

(* Fonction pour calculer le cardinal de l'ensemble L(m) *)
let cardinal_L m = longueur (ensemble_L m)
```

Générations de graphes aléatoires

Étant donnés deux nombres N et M, on définit le graphe $G(N,M)$ comme le graphe qui a pour sommets l'ensemble $[\![0,N-1]\!]$ et pour arêtes l'ensemble $\{\{l_i,g_i\}|0\leq i<M\}$ où l_i et g_i sont définis de la façon suivante :

- $l_i = u_{2i} \mod N$
- $g_i = (l_i + 1 + (u_{2i+1} \mod (N-1))) \mod N$

Correction :
Pour les questions suivantes, j'ai décidé de commencer en implémentant une fonction `graphe : int -> int -> int list array` qui, étant donnés N et M, renvoie $G(N,M)$ sous forme de listes d'adjacence. Pour cela, on calcule les l_i et g_i. Pour chacun d'entre eux, on ajoute l'autre dans la liste d'adjacence de l'un. Enfin, on élimine les doublons dans les listes d'adjacence à l'aide de la fonction précédente.

```
(* Fonction pour générer un graphe avec n sommets et m arêtes *)
let graphe n m =
  (* Tableau où chaque sommet est associé à la liste de ses voisins *)
  let voisins = Array.make n [] in
  (* Fonction récursive pour remplir les arêtes du graphe *)
  let rec remplir i = match i with
    | -1 -> ()  (* Arrêt quand on a traité toutes les arêtes *)
    | i -> let l_i = t.(2 * i) mod n in  (* Calcul de l_i)
             let g_i = (l_i + 1 + (t.(2 * i + 1) mod (n - 1))) mod n in
                 (* Calcul de g_i *)
             (* Ajoute les arêtes bidirectionnelles entre l_i et g_i *)
             voisins.(g_i) <- l_i::(voisins.(g_i));
             voisins.(l_i) <- g_i::(voisins.(l_i));
             remplir (i-1)
  in remplir (m-1);  (* Démarre la récursion en ajoutant les arêtes *)
  (* Éliminer les doublons dans les listes de voisins *)
  for i = 0 to n-1 do
    voisins.(i) <- elimineDoublon (voisins.(i));
  done;
  voisins  (* Retourne le graphe *)
```

Question 3 : Calculer le nombre de voisins du sommet 0 dans les graphes $G(N,M)$ pour les valeurs de N et M suivantes :

- $N = 5, M = 10$
- $N = 100, M = 300$
- $N = 1\,234, M = 123\,456$
- $N = 98\,765, M = 234\,567$

Correction :
Une fois le graphe construit, il suffit de calcule la longueur de la liste d'adjacence de 0.

```
let degre g s = longueur (g.(s))
```

Question 4 : Calculer le nombre total d'arêtes des graphes $G(N,M)$ pour les valeurs de N et M suivantes :

- $N = 5, M = 10$
- $N = 100, M = 300$
- $N = 1\,234, M = 123\,456$
- $N = 98\,765, M = 234\,567$

Correction :
Pour compter le nombre d'arêtes, on peut calculer la somme des degrés des sommets à l'aide de la fonction précédente puis la diviser par deux pour obtenir le résultat recherché.

```
(* Fonction pour calculer le nombre d'arêtes dans un graphe g *)
let nb_aretes g =
  let somme = ref 0 in
  (* Parcourt tous les sommets et accumule leurs degrés *)
  for i = 0 to Array.length g -1 do
    somme := !somme + degre g i;
  done;
  (* Le nombre d'arêtes est la moitié de la somme des degrés *)
  !somme/2
```

Parcours dans les graphes

Un graphe non orienté $G = (S, A)$ est dit connexe si quels que soient les sommets u et v de A, il existe un chemin reliant u à v. Une composante connexe d'un graphe G est un sous-graphe connexe de ce graphe G.

Question 5 : Calculez le nombre de sommets appartenant à la composante connexe du sommet 0 pour chaque graphe $G(N, M)$, avec les valeurs de N et M suivantes :

- $N = 5, M = 10$
- $N = 100, M = 300$
- $N = 1\,234, M = 123\,456$
- $N = 98\,765, M = 234\,567$

Correction :
Pour calculer le nombre de sommets dans la composante connexe de 0 dans G, on doit réaliser un parcours depuis ce sommet. Le plus simple à implémenter est le parcours en profondeur, c'est donc le choix que j'ai fait. Une fois le parcours fini, il suffit de compter le nombre de cases du tableau `vus` ayant pour valeur `true` pour connaître le nombre de sommets visités.

```
(* Fonction pour parcourir le graphe et marquer les sommets visités *)
let rec parcours g l vus = match l with
  | [] -> ()  (* Si la liste est vide, on arrête *)
  | h::q when vus.(h) -> parcours g q vus  (* Si le sommet h est déjà
     visité, on continue *)
  | h::q ->
    vus.(h) <- true;  (* Marque le sommet h comme visité *)
    parcours g (g.(h)) vus;  (* Parcourt les voisins du sommet h *)
    parcours g q vus  (* Continue à parcourir le reste de la liste *)

(* Fonction pour calculer la taille de la composante connexe contenant
   un sommet s *)
let rec taille_composante_connexe g s =
  let vus = Array.make (Array.length g) false in
  vus.(s) <- true;  (* Marque le sommet s comme visité *)
  parcours g (g.(s)) vus;  (* Parcourt la composante connexe à partir
     de s *)
  (* Compte les sommets visités *)
  let rec compter i = match i with
    | -1 -> 0
    | i when vus.(i) -> 1 + compter (i-1)
    | i -> compter (i-1)
  in compter (Array.length g - 1)
```

Question 6 : Calculez le nombre de composantes connexes pour chaque graphe $G(N, M)$, avec les valeurs de N et M suivantes :

- $N = 5, M = 10$
- $N = 100, M = 300$
- $N = 1\,234, M = 123\,456$
- $N = 98\,765, M = 234\,567$

Correction :
Comme précédemment, on réalise des parcours en profondeur. On relance un parcours à chaque fois que l'on trouve un sommet que l'on a pas encore visité.

```
(* Fonction pour calculer le nombre de composantes connexes dans un
   graphe g *)
let nombre_composantes g =
  let vus = Array.make (Array.length g) false in
  let nb = ref 0 in
  (* Parcourt tous les sommets du graphe *)
  for i = 0 to Array.length g - 1 do
    if not vus.(i) then  (* Si un sommet n'est pas encore visité *)
      begin
        vus.(i) <- true;  (* Marque le sommet comme visité *)
        parcours g (g.(i)) vus;  (* Parcourt sa composante connexe *)
        nb := !nb + 1  (* Incrémente le nombre de composantes *)
      end
  done;
  !nb  (* Retourne le nombre de composantes connexes *)
```

Coloriage des graphes

Un « recoloriage » d'un sommet s dans un coloriage consiste à remplacer la couleur du sommet s par le plus petit entier naturel qui n'est pas utilisé par un des voisins de s. Un premier algorithme, pour colorier les graphes consiste simplement à partir du coloriage où tous les sommets sont coloriés à 0 puis à recolorier chacun des sommets, en commençant par le sommet 0, puis le sommet 1, puis le sommet 2, etc., jusqu'au sommet $N-1$.

Correction :
Pour les deux questions suivantes, j'ai commencé par implémenter la fonction de recoloriage comme expliqué.

```
(* Fonction pour trouver la plus petite couleur disponible pour un
   sommet *)
let couleur_dispo l =
  let rec aux i l1 = match l1 with
    | [] -> i  (* Si la liste est vide, retourne i *)
    | h::q when h = i -> aux (i + 1) l  (* Si la couleur i est prise,
       teste la suivante *)
    | h::q -> aux i q  (* Sinon, continue à chercher dans la liste *)
  in aux 0 l

(* Fonction pour colorier un graphe *)
let recolorier g =
  let coloriage = Array.make (Array.length g) 0 in
  (* Pour chaque sommet, attribue la couleur la plus petite non utilisé
     e par ses voisins *)
  for i = 0 to (Array.length g - 1) do
    coloriage.(i) <- couleur_dispo (List.map (fun x -> coloriage.(x)) g
       .(i));
  done;
  coloriage  (* Retourne le tableau de coloriage *)
```

Question 7 : Dans le coloriage produit par cette méthode sur les graphes $G(N,M)$, quelle est la couleur du sommet $\lfloor N/2 \rfloor$ pour les valeurs de N et M suivantes :

- $N=5, M=10$
- $N=100, M=300$
- $N=1\,234, M=123\,456$
- $N=98\,765, M=234\,567$

Correction :
On applique la fonction `recolorier` sur le graphe et on renvoie la couleur du sommet étudié.

```
(* Fonction pour obtenir la couleur du sommet central du graphe *)
let couleur_sommet_central g =
  let coloriage = recolorier g in
  coloriage.(Array.length g / 2)  (* Retourne la couleur du sommet au
     milieu du graphe *)
```

Question 8 : Dans le coloriage produit par cette méthode sur les graphes $G(N,M)$, quel est le nombre total de couleurs utilisées pour les valeurs de N et M suivantes :

- $N = 5, M = 10$
- $N = 100, M = 300$
- $N = 1\,234, M = 123\,456$
- $N = 98\,765, M = 234\,567$

Pour connaître le nombre de couleurs utilisées, il suffit de connaître le numéro maximal utilisé pour une couleur. On doit ainsi seulement ajouter 1 puisque les numéros de couleurs commencent à 0.

```
(* Fonction pour compter le nombre de couleurs utilisées dans un
   graphe colorié *)
let nombre_couleurs g =
  let coloriage = recolorier g in
  let nb = ref 0 in
  (* Parcourt le graphe pour trouver la couleur maximale *)
  for i = 0 to Array.length g - 1 do
    if coloriage.(i) > !nb then nb := coloriage.(i);
  done;
  !nb+1  (* Retourne le nombre de couleurs utilisées *)
```

On note $P(N,M,K)$ comme la chaîne de caractère $b_0 b_1 b_2 \cdots b_9$ où $b_i = 1$ quand $G(N+i, M+i)$ est coloriable avec K couleurs et $b_i = 0$ sinon.

Question 9 : Calculer $P(N,M,2)$ pour les valeurs de N et M suivantes :

- $N = 10, M = 3$
- $N = 1\,234, M = 90$
- $N = 98\,765, M = 1\,000$

Correction :
Pour tester si un graphe est bicoloriable, il suffit de réaliser un parcours en profondeur. En effet, lorsque l'on fixe la couleur d'un sommet, celle de ses voisins est forcée. Il en sera alors de même pour les voisins de ses voisins et ainsi de suite.

```
(* Fonction pour vérifier si un graphe est bicolorable *)
let est_bicolorable n m =
  let g = graphe n m in
  let couleurs = Array.make n (-1) in
  (* Fonction récursive pour colorier les sommets *)
  let rec bicolorer l c = match l with
    | [] -> true
    | h::q when couleurs.(h) = -1 ->
      couleurs.(h) <- (1 - c);  (* Assigne une couleur différente à
          chaque voisin *)
      bicolorer (g.(h)) (1 - c) && bicolorer q c
    | h::q -> couleurs.(h) = 1 - c
  in
  let bicoloriable = ref true in
  (* Parcourt les sommets du graphe et essaie de les colorier *)
  for i = 0 to Array.length g - 1 do
    bicoloriable :=
      (couleurs.(i) != -1 || (couleurs.(i) <- 0; bicolorer (g.(i)) 0))
          && (!bicoloriable);
  done;
  !bicoloriable  (* Retourne true si le graphe est bicolorable *)

(* Fonction qui calcule P(N,M,2) *)
let p_n_m_2 n m =
  let rec aux i acc =
    if i = 10 then acc  (* Limite à 10 essais *)
    else
      let b_i = if est_bicolorable (n + i) (m + i) then '1' else '0' in
      aux (i + 1) (acc ^ (String.make 1 b_i))  (* Concatène les ré
          sultats de bicolorabilité *)
  in aux 0 ""  (* Démarre la récursion avec un résultat vide *)
```